Local Randomized Neural Networks with Discontinuous Galerkin Methods for Partial Differential Equations

Summary of Sun, Dong, and Wang (2022)

1 Introduction

This paper introduces novel computational methods that combine local randomized neural networks (LRNN) with discontinuous Galerkin (DG) approaches for solving partial differential equations (PDEs). The key innovation lies in using randomized neural networks where hidden-layer parameters are fixed to randomly assigned values and only output-layer parameters are computed by solving a linear system through least squares. This approach maintains accuracy while significantly improving computational efficiency.

The methods address several limitations of traditional neural network-based approaches:

- The high computational cost of training fully parameterized neural networks
- Accuracy limitations of physics-informed neural networks
- The difficulty of handling problems with low regularity

The proposed framework introduces three schemes:

- LRNN-DG: Local randomized neural networks with discontinuous Galerkin
- LRNN-C^0DG: Local randomized neural networks with C^0 discontinuous Galerkin
- LRNN-C^1DG: Local randomized neural networks with C^1 discontinuous Galerkin

These methods combine domain decomposition with neural network approximation, using outputs from local neural networks' last hidden layers as basis functions and applying DG formulations to connect them across subdomain boundaries.

2 Algorithm Description

2.1 Randomized Neural Networks Structure

The randomized neural network structure is defined as follows:

- Parameters of hidden layers are randomly assigned and fixed during training
- Only the weights for the output layer are trained using a least-squares method
- Single-hidden-layer networks are used with one-dimensional output

The function space for randomized neural networks is defined as:

$$M_{RNN}(D) = \left\{ U(\beta, \theta, x) = \sum_{j=1}^{M} \beta_j \phi_j^D(x) : x \in D \right\} \tag{1}$$

where $D \subset \mathbb{R}^n$ is the domain, M is the width of the last hidden layer, θ represents fixed parameters of hidden layers, β represents trainable parameters of the output layer, and $\phi_j^D(x)$ is the output of the last hidden layer.

2.2 LRNN-DG Method

Using the Poisson problem as a model:

$$-\Delta u = f \quad \text{in } \Omega \tag{2}$$

$$u = g \quad \text{on } \partial\Omega \tag{3}$$

The domain is partitioned into subdomains with a local neural network assigned to each. The DG space is defined as:

$$V_h = \{ v_h \in L^2(\Omega) : v_h|_K \in M_{RNN}(K) \, \forall K \in \mathcal{T}_h \} \tag{4}$$

where $\mathcal{T}_h$ is the decomposition of the domain.

The LRNN-DG method formulation is: Find $u_h \in V_h$ such that

$$a_h(u_h, v_h) = l(v_h) \quad \forall v_h \in V_h \tag{5}$$

where

$$a_h(u, v) = \int_\Omega \nabla_h u \cdot \nabla_h v \, dx - \int_{\mathcal{E}_h} \{r_h u\} \cdot v \, ds - \int_{\mathcal{E}_h} u \cdot \{\nabla_h v\} \, ds + \int_{\mathcal{E}_h} \sigma u \cdot v \, ds \tag{6}$$

$$l(v_h) = \int_\Omega f v_h \, ds - \int_{\mathcal{E}_h^\partial} g n \cdot \nabla_h v_h \, ds + \int_{\mathcal{E}_h^\partial} \sigma g v_h \, ds \tag{7}$$

The resulting system of equations is:

$$AU = L \tag{8}$$

where A is an $N_eM \times N_eM$ matrix, L is an $N_eM \times 1$ vector, N_e is the number of elements, and M is the width of the last hidden layer.

2.3 LRNN-C^{0}DG Method

The LRNN-C^{0}DG method enforces continuity conditions across subdomain boundaries through collocation points, eliminating the need for penalty parameters. It adds constraints:

1. Boundary conditions at collocation points on boundary edge:

$$u_h(x_j^e) = g(x_j^e) \quad \forall x_j^e \in \mathcal{P}_h^\partial \tag{9}$$

2. C^0-continuity conditions at collocation points on interior edges:

$$u_h(x_j^e) = 0 \quad \forall x_j^e \in \mathcal{P}_h^i \tag{10}$$

The LRNN-C^{0}DG scheme becomes: Find $u_h \in V_h$ such that

$$a_h^0(u_h, v_h) = l^0(v_h) \quad \forall v_h \in V_h \tag{11}$$

$$u_h(x_j^e) = 0 \quad \forall x_j^e \in \mathcal{P}_h^i \tag{12}$$

$$u_h(x_j^e) = g(x_j^e) \quad \forall x_j^e \in \mathcal{P}_h^\partial \tag{13}$$

This leads to the augmented system:

$$\begin{bmatrix} A_1 \\ A_2 \end{bmatrix} U = \begin{bmatrix} L_1 \\ L_2 \end{bmatrix} \tag{14}$$

solved using least squares.

2.4 LRNN-C^{1}DG Method

The LRNN-C^{1}DG method enforces both C^0 and C^1 continuity conditions:

1. C^0-continuity (function value continuity): $u_h(x_j^e) = 0$
2. C^1-continuity (gradient continuity): $[\nabla u_h(x_j^e)] = 0$
3. Boundary conditions: $u_h(x_j^e) = g(x_j^e)$

The method is formulated as: Find $u_h \in V_h$ such that

$$a_h^K(u_h, v_h) = \int_K f v_h \, dx \quad \forall v_h \in V_h \quad \forall K \in \mathcal{T}_h \tag{15}$$

$$u_h(x_j^e) = 0 \quad \forall x_j^e \in \mathcal{P}_h^i \tag{16}$$

$$[\nabla u_h(x_j^e)] = 0 \quad \forall x_j^e \in \mathcal{P}_h^i \tag{17}$$

$$u_h(x_j^e) = g(x_j^e) \quad \forall x_j^e \in \mathcal{P}_h^\partial \tag{18}$$

This results in a larger system of equations solved using least squares.

2.5 Space-Time LRNN-DG Methods for Time-Dependent Problems

For time-dependent problems like the heat equation:

$$u_t(t,x) - \Delta u(t,x) = f(t,x) \quad \text{in } I \times \Omega \tag{19}$$

$$u(0,x) = u_0(x) \quad \text{in } \Omega \tag{20}$$

$$u(t,x) = g(t,x) \quad \text{on } I \times \partial\Omega \tag{21}$$

The space-time approach treats temporal and spatial variables equally, avoiding error accumulation from time-stepping:

1. The domain $I \times \Omega$ is partitioned into space-time elements
2. Local neural networks approximate the solution on each space-time element
3. DG formulations couple the elements together
4. Three variants (LRNN-DG, LRNN-C^0DG, LRNN-C^1DG) are extended to space-time

The space-time LRNN-DG scheme is formulated as: Find $u_h^\tau \in V_h^\tau$ such that

$$B_h^\tau(u_h^\tau, v_h^\tau) = l(v_h^\tau) \quad \forall v_h^\tau \in V_h^\tau \tag{22}$$

The resulting system is solved in one step rather than through time-marching.

3 Theoretical Findings

3.1 Convergence Analysis for LRNN-DG Method

The key theoretical properties established include:

1. **Consistency**: The LRNN-DG scheme is consistent with the original PDE.
2. **Boundedness**: The bilinear form $a_h(u,v)$ satisfies the boundedness condition:
$$a_h(u,v) \le C_b \|u\|_* \|v\|_* \quad \forall v \in V(h) \tag{23}$$
3. **Stability**: With sufficient penalty parameter σ_0, the bilinear form satisfies:
$$a_h(v,v) \ge C_s \|v\|_*^2 \quad \forall v \in V_h \tag{24}$$

4. **Céa-type Inequality**: The error between the exact solution u and the numerical solution u_h is bounded by:

$$\|u - u_h\|_* \leq (1 + C_b/C_s) \inf_{v_h \in V_h} \|u - v_h\|_* \tag{25}$$

5. **Approximation Property**: Under appropriate assumptions about the approximation capabilities of neural networks, for any small positive ϵ, there exists a positive integer M_ϵ such that if $M > M_\epsilon$, then:

$$\|u - u_h\|_* \leq C\epsilon \tag{26}$$

3.2 Convergence Analysis for LRNN-C^{0}DG and LRNN-C^{1}DG Methods

For LRNN-C^{0}DG and LRNN-C^{1}DG methods, convergence requires additional assumptions:

1. **Jump Reduction**: As the number of collocation points increases, the jumps in function values and derivatives decrease. For LRNN-C^{0}DG:

$$\|\tilde{u}_h\|_{0,e} \leq C h_e^{1/2} \epsilon \quad \text{and} \quad \|\tilde{u}_h - g\|_{0,e} \leq C h_e^{1/2} \epsilon \tag{27}$$

   For LRNN-C^{1}DG, additional gradient jump reduction:

$$\|[\nabla u_h(x_j^e)]\|_{0,e} \leq C h_e^{-1/2} \epsilon \tag{28}$$

2. **Convergence Results**: Under these assumptions, both methods achieve

$$\|u - u_h\|_* \leq C\epsilon \tag{29}$$

   for sufficiently large M and number of collocation points N_e.

4 Numerical Examples and Results

4.1 One-Dimensional Helmholtz Equation

For the problem:

$$u_{xx} - \lambda u = f(x) \quad \text{in } \Omega = [0, 1] \tag{30}$$
$$u(0) = \sin(0.4\pi)\cos(0.4\pi) \tag{31}$$
$$u(1) = \sin(4.4\pi)\cos(4.4\pi) \tag{32}$$

with exact solution $u(x) = \sin(4\pi(x + 0.1))\cos(4\pi(x + 0.1))$

Results show that:

- All three methods (LRNN-DG, LRNN-C^0DG, LRNN-C^1DG) achieve high accuracy
- With just 80 degrees of freedom (DoF) per subdomain, errors in L^2 norm reach the order of 10^{-9}
- LRNN-C^1DG slightly outperforms LRNN-C^0DG, which outperforms LRNN-DG
- For fixed DoF per subdomain, errors decrease as the subdomain size decreases
- For fixed subdomain size, errors decrease rapidly with increasing DoF initially, then stabilize

4.2 Two-Dimensional Poisson Equation

For the problem:

$$-\Delta u = f \quad \text{in } \Omega = [0,1]^2 \tag{33}$$
$$u = g \quad \text{on } \partial\Omega \tag{34}$$

with exact solution $u = e^{x+y}\sin(3\pi x + 0.5\pi)\cos(\pi y + 0.2\pi)$

Results show that:

- All three methods achieve high accuracy, with LRNN-C^1DG performing best
- With 160 DoF per element, L^2 errors reach 10^{-6} to 10^{-8}
- Comparison with traditional finite element method (FEM) and DG method shows superior accuracy per DoF
- LRNN methods outperform polynomial-based methods (P_k FEM and P_k DG) with the same DoF
- Error distribution analysis shows LRNN-DG produces smoother but larger errors, while LRNN-C^0DG and LRNN-C^1DG have smaller but more oscillatory errors

4.3 Heat Equation (Time-Dependent Problem)

For the problem:

$$u_t(t,x) - \lambda\Delta u(t,x) = f(t,x) \quad \text{in } I \times \Omega \tag{35}$$
$$u(0,x) = u_0(x) \quad \text{in } \Omega \tag{36}$$
$$u(t,x) = g(t,x) \quad \text{on } I \times \partial\Omega \tag{37}$$

where $\Omega = [0, 1]$, $I = [0, 1]$, and $\lambda = 1$ or 0.001, with exact solution $u = -e\cos(\pi x + 3\pi) + t^2$

Results from space-time methods show:

- All space-time LRNN methods achieve high accuracy in a single computation step
- With 320 DoF per space-time element, L^2 errors reach 10^{-7} to 10^{-8}
- Space-time LRNN methods outperform traditional FEM with backward Euler time-stepping
- Error distribution analysis shows traditional methods suffer from error accumulation over time, while space-time LRNN methods avoid this issue
- LRNN-C^0DG and LRNN-C^1DG provide better accuracy than LRNN-DG

The numerical tests also validate theoretical assumptions about jump reduction with increasing collocation points, showing that jumps in function values and derivatives become negligible (near machine precision) as the number of collocation points increases.

5 Implications and Advantages

5.1 Computational Efficiency

- **Reduced Parameter Count**: By training only output layer weights, LRNN methods drastically reduce the number of trainable parameters compared to fully parameterized neural networks
- **Linear System Advantage**: The training reduces to solving a linear system rather than non-convex optimization, improving both efficiency and stability
- **Space-Time Approach**: For time-dependent problems, the space-time formulation allows solving the entire problem in one step, avoiding time-stepping and error accumulation
- **Mesh-Free Nature**: The methods are effectively mesh-free for local approximation, making them well-suited for complex geometries

5.2 Accuracy and Solution Quality

- **Superior Accuracy per DoF**: The methods achieve better accuracy than traditional FEM and DG methods with the same number of degrees of freedom

- **Penalty Parameter Freedom**: LRNN-C^0DG and LRNN-C^1DG eliminate the need for penalty parameters, avoiding the challenge of parameter tuning
- **Collocation Effectiveness**: The experiments confirm that using collocation points to enforce continuity conditions is highly effective
- **Gradient Accuracy**: For problems requiring derivative information, LRNN-C^1DG provides high-quality gradient approximations

5.3 Versatility and Adaptability

- **Framework Flexibility**: The approach combines the advantages of neural networks (approximation power) with the mathematical foundation of DG methods (well-established theoretical framework)
- **Problem Scope**: The methods handle both steady-state and time-dependent problems effectively
- **Boundary Condition Handling**: Different types of boundary conditions are naturally incorporated
- **Multiple Formulations**: The three formulations (LRNN-DG, LRNN-C^0DG, LRNN-C^1DG) offer options for different problem requirements

5.4 Potential Limitations and Future Directions

- **Error Stagnation**: The errors appear to stagnate after reaching a certain threshold, possibly due to linear dependence among basis functions with larger network widths
- **Extension to Nonlinear Problems**: The current formulation addresses linear PDEs, with extension to nonlinear problems identified as a future direction
- **Parallel Processing Potential**: The local nature of the approximation suggests good parallelization potential
- **Activation Function and Random Distribution**: The choice of activation functions and parameter distributions affects performance and warrants further investigation